

What shape is in the image? / How many sides does the shape in the image have?						
Model	Triangle	Square	Pentagon	Hexagon	Heptagon	Octagon
LLaVA-1.5	96% / 93%	99% / 100%	0% / 0%	5% / 90%	0% / 0%	0% / 3%
LLaVA-Next	100% / 99%	100% / 98%	0% / 0%	94% / 0%	0% / 0%	26% / 99%
LLaVA-OneVision	100% / 100%	100% / 100%	54% / 72%	100% / 100%	0% / 0%	100% / 99%
Qwen2-VL	100% / 100%	100% / 100%	97% / 99%	92% / 88%	14% / 3%	99% / 97%
LLaMA-3.2	100% / 98%	99% / 78%	85% / 70%	94% / 96%	0% / 9%	42% / 20%
InternVL	100% / 100%	98% / 100%	0% / 80%	89% / 77%	0% / 0%	5% / 3%
Molmo	100% / 100%	99% / 100%	47% / 7%	93% / 94%	55% / 51%	87% / 94%
Janus-Pro	100% / 100%	100% / 100%	100% / 100%	100% / 100%	0.6% / 4%	65% / 77%
Math-LLaVA	100% / 86%	100% / 100%	42% / 79%	78% / 61%	0% / 0%	87% / 90%
G-LLaVA	93% / 12%	50% / 92%	0% / 8%	55% / 92%	0% / 0%	0% / 9%
Math-PUMA	100% / 100%	100% / 100%	26% / 18%	100% / 100%	0% / 0%	0% / 0%
GPT-4-Turbo	100% / 98%	98% / 99%	87% / 85%	100% / 95%	0.1% / 10%	69% / 78%
GPT-4o	99% / 100%	97% / 100%	100% / 100%	68% / 78%	92% / 58%	49% / 71%

Table 1: **Model accuracy for shape identification (first value) and side counting (second value).** Notably, models struggle to count sides accurately. GPT-4o identifies heptagons with 92% accuracy but counts sides correctly only 58% of the time, revealing a gap between recognition and reasoning.

only GPT-4o and Molmo exceeding 1% accuracy. Notably, Molmo is trained with a point-then-count approach, identifying objects before quantifying them. Despite this targeted training, Molmo correctly counts pentagon sides only 7% of the time and heptagons 51% of the time (more examples of Molmo behavior in Appendix F). Moreover, GPT-4o correctly identifies heptagons 92% of the time but counts their sides accurately only 58% of the time, revealing its tendency to recognize shapes without using visual information to reason about their properties. These findings prompt a key question: is the breakdown in shape recognition and side-counting due to flawed visual processing or a failure to apply reasoning? We explore this by analyzing the vision encoder and LLM backbone separately.

3.1.1 Which visual features affect accuracy?

To better understand the causes of model errors, we analyzed the relationship between accuracy and five visual attributes: background color, shape color, size, rotation, and contrast ratio. The contrast ratio is defined as the difference between the background and shape colors. Table 2 shows the correlation coefficients between each factor and model accuracy for thirteen different models. A positive correlation means that accuracy improves as the factor increases, while a negative correlation suggests the factor has a harmful effect on accuracy.

The results show that no single visual factor explains performance across all models. For example, rotation has a strong negative effect in Qwen2-VL and Math-LLaVA but has almost no effect in LLaVA-OneVision. On average, rotation shows the

Model	Background Color	Shape Color	Size	Rotation	Contrast Ratio
LLaVA 1.5	-0.005	0.066	0.126	0.089	0.047
LLaVA 1.6	-0.197	-0.015	0.039	0.010	-0.110
Qwen-VL	0.127	0.009	0.096	-0.226	0.062
InternVL	-0.042	0.061	0.186	0.011	-0.065
LLaVA-OneVision	0.012	0.033	0.026	-0.001	-0.020
LLaMA 3.2	0.021	0.026	0.049	0.010	0.014
Molmo	0.003	0.020	0.039	-0.270	-0.005
Janus	-0.184	0.008	0.223	-0.144	-0.146
GPT-4-Turbo	-0.056	0.051	0.035	-0.092	-0.023
GPT-4o	0.031	0.039	-0.054	-0.144	0.068
Math-LLaVA	0.122	-0.001	-0.006	-0.362	0.066
G-LLaVA	0.209	0.060	0.023	-0.040	0.137
Math-PUMA	0.018	-0.017	0.172	-0.079	-0.020
Abs Val Avg	0.079	0.031	0.083	0.098	0.060

Table 2: Correlation between visual features and model accuracy. Values represent how strongly each factor relates to correct predictions. Rotation and size have the highest average influence.

strongest overall influence, followed by size.

Contrast ratio, a factor that is often important in traditional vision tasks, shows mixed results. GPT-4o and G-LLaVA perform slightly better with higher contrast, while LLaVA 1.6 and Janus perform worse. These inconsistencies suggest that visual sensitivity varies significantly depending on model architecture and training.

Taken together, these findings suggest that misclassifications are not caused by a single weak point in the visual input. Instead, each model exhibits its own unique set of sensitivities. These insights complement our broader evaluation of shape recognition and motivate a closer look at the components of the system. In the next sections, we analyze the language model and the vision encoder independently to understand their contributions to these errors.

Name the $\langle n \rangle$ -sided polygon / Num of sides in $\langle shape \rangle$	
LM Only	Accuracy (%)
LLaVA-1.5	67% / 100%
LLaVA-Next	100% / 100%
LLaVA-OneVision	100% / 100%
Qwen2-VL	100% / 100%
LLaMA-3.2	100% / 100%
InternVL	100% / 100%
Molmo	100% / 100%
Janus-Pro	100% / 100%
Math-LLaVA	100% / 100%
G-LLaVA	50% / 100%
Math-PUMA	100% / 100%
GPT-4o	100% / 100%
GPT-4-Turbo	100% / 100%

Table 3: Accuracy of the LLM-backbone on the polygon naming and side counting tasks: “What is the name of a $\langle n \rangle$ -sided regular polygon? / How many sides does $\langle shape \rangle$ have?” for $n \in \{3, 4, \dots, 8\}$ and shape in $\{\text{triangle, square, } \dots, \text{octagon}\}$.

3.1.2 What does the text-decoder know?

To assess LLMs’ knowledge of the geometric properties MLLMs struggle with, we construct the following prompts: (1) “What is the name of a $\langle n \rangle$ -sided polygon?” where $\langle n \rangle$ ranges from 3 to 8, and (2) “How many sides does a $\langle shape \rangle$ have?” where $\langle shape \rangle$ is a regular polygon from triangle to octagon. The results, shown in Table 9, reveal that the underlying language models know geometric properties. All models get 100% accuracy on “How many sides does a $\langle shape \rangle$ have?”, and 11 out of 13 models get 100% accuracy on “What is the name of a $\langle n \rangle$ -sided polygon?”. The success of LLM backbones in these tasks aligns with prior findings that state-of-the-art models perform well on lower-level math problems (Xiang et al., 2025). Note that LLaVA 1.5 and G-LLaVA’s LMs struggle with naming certain shapes, suggesting some failures stem from gaps in pretraining data rather than vision-related issues. In Appendix E, we analyze Google N-Grams data and confirm that the shapes these models struggle with are also the least frequently mentioned in text.

3.1.3 What does the vision-encoder see?

To evaluate how vision encoders process shapes, we use t-SNE to visualize their embeddings. Figure 2 shows results for LLaVA-OneVision, which exhibits the same embedding pattern as all other models (Appendix D). Common shapes like triangles (blue) and squares (red) form well-defined clusters, reflecting the models’ strong performance

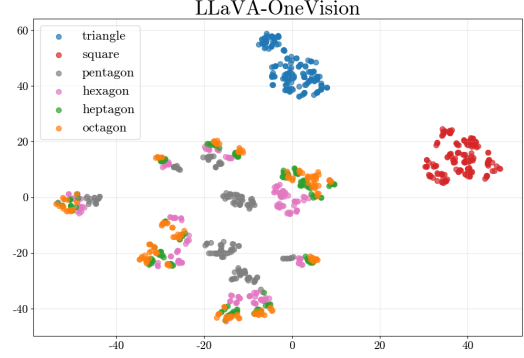


Figure 2: **T-SNE plots of vision encoder embeddings from LLaVA-OneVision.** Only triangles and squares form distinct clusters. Appendix D shows all models learn a similar embedding.

on these shapes (Table 1). In contrast, vision encoders appear “shape-blind” to less common shapes, often embedding them into the same cluster. In particular, hexagons (pink), heptagons (green), and octagons (orange) exhibit dispersed and overlapping embeddings indicating poor differentiation. Appendix D provides nearest neighbor analysis for all models, confirming that triangles and squares form distinct clusters in vision-encoder representations. In LLaVA-OneVision, 99.1% of a triangle’s 20 nearest neighbors are also triangles, while only 41.1% of a heptagon’s neighbors are heptagons, with most being hexagons or octagons. This trend is consistent across all open-source models.

Although LLMs excel at answering basic shape property questions, vision encoders in MLLMs struggle to differentiate shapes, causing the entire MLLM system to misidentify shapes. The contrast between Table 1 and Table 9 suggests vision encoder limitations as a key failure, aligning with prior findings on their lack of fine-grained detail (Nayak et al., 2022). The interplay between weak vision encoders and strong LLM backbones is evident in GPT-4o and GPT-4 Turbo. GPT-4o outperforms GPT-4-Turbo in shape identification, raising heptagon identification from 0% to 92%, yet struggles with side counting (58% accuracy). While a human would engage in System 2 reasoning and count the sides (e.g., “1, 2, 7” → heptagon) when faced with an unfamiliar shape, models instead default to predicting 6 or 8 sides, suggesting reliance on memorized patterns (System 1) rather than genuine visual reasoning. These findings highlight a fundamental issue: recognizing shapes in pretraining data does not generalize to an understanding of geometric properties such as having

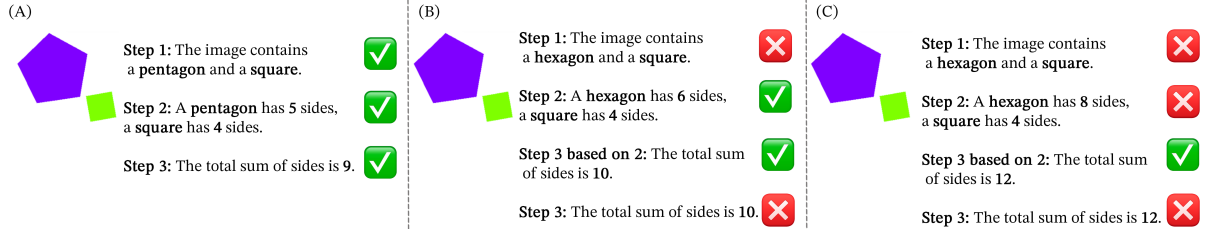


Figure 3: **Illustration of failure modes in the two-shape reasoning task.** A: Successful completion of all steps. B: The most common failure mode, where misidentification in Step 1 leads to an incorrect sum in Step 3. C: An error in mapping shapes to their number of sides (Step 2), affecting the final sum.

“sides”. This disconnect prompts an examination of how the lack of visual understanding of “sides” impacts performance on more complex, multi-step reasoning tasks, which more closely resemble real-world math datasets.

3.2 Evaluating Multi-step Math Reasoning

1. Identify the shapes, 2. Map shape to side, 3. Sum all sides.				
VLM Accuracy	Step 1	Step 2	Step 3 from 2	Step 3
LLaVA-1.5	8.92%	96.94%	87.50%	11.61%
LLaVA-Next	14.09%	99.80%	97.76%	26.33%
LLaVA-OneVision	53.38%	99.98%	99.96%	43.61%
Qwen2-VL	71.71%	100.00%	98.70%	55.15%
LLaMA-3.2	2.67%	99.33%	38.25%	31.02%
InternVL	41.63%	99.88%	98.13%	43.54%
Molmo	27.92%	97.35%	99.92%	27.27%
Janus-Pro	50.77%	95.05%	63.26%	52.20%
Math-LLaVA	0.00%	0.06%	0.00%	14.18%
G-LLaVA	0.00%	0.00%	0.00%	3.81%
Math-PUMA	32.08%	100.00%	78.29%	33.32%
GPT-4-Turbo	52.95%	99.96%	81.73%	50.59%
GPT-4o	73.40%	99.98%	97.74%	56.23%

Table 4: **Step-wise accuracies.** Although models incorrectly identify the shapes, they still perform the correct mapping of shapes to their respective sides and then correctly sum. See Figure 3.

We use a three-step mathematical reasoning pipeline to pinpoint bottlenecks in MLLM geometric reasoning: Step 1: identifying shapes in an image; Step 2: retrieving their side counts; and Step 3: summing the sides of two shapes. Combining multi-shape recognition with arithmetic operations helps separate vision challenges (e.g., shape recognition) from reasoning limitations (e.g., arithmetic errors or mapping mistakes). Following the single-shape generation procedure (Appendix B.2), we extend the dataset to include two-shape images with variations in color, background, size, and rotation, ensuring no overlap or collision. Table 4 reveals that most models struggle in the two-shape setting, with the shape identification step (Step 1) emerging as the primary bottleneck. While models like GPT-4o and GPT-4 Turbo achieve near-perfect accuracy

in recalling shape properties (Step 2), their failure to correctly recognize shapes in Step 1 limits overall performance. For instance, GPT-4o identifies shapes with 73.40% accuracy but achieves only 56.23% accuracy in Step 3. Similarly, LLaVA-Next, InternVL, LLaVA-OneVision, and Molmo perform well in mapping and arithmetic reasoning but frequently misidentify shapes, leading to errors seen in Figure 3, panel B. In addition to the ground-truth sum of sides, we evaluate accuracy for Step 3 based on the output of Step 2, allowing us to isolate errors in shape identification from arithmetic errors (see Figure 3). Consider an image containing a pentagon and a square. If the model misidentifies the pentagon as a hexagon, Step 3 accuracy would still be correct if the sum aligns with the model’s incorrect mapping (e.g., $6 + 4 = 10$ instead of the correct $5 + 4 = 9$). Table 4 demonstrates that models execute sums correctly based on the side mapping from Step 2. Namely, the low performance in Step 3 accuracies is primarily due to an inability to retrieve the correct shapes in the image, not an inability to execute sums.

Examining reasoning through distinct steps highlights a fundamental limitation in multimodal integration. MLLMs succeed in basic arithmetic but fail when vision and language must work together. Their tendency to rigidly map less common shapes to familiar ones (e.g., misclassifying heptagons as hexagons) highlights the lack of reliable visual grounding. Even with strong LLM backbones, this limitation persists, reflecting broader challenges in vision-language mathematical benchmarks (Zhang et al., 2024a). To investigate this further, we evaluate performance on entirely unseen shapes devoid of familiar visual priors and biases.

3.3 Counting Sides of Abstract Shapes

We introduce a dataset of novel shapes to test the hypothesis that MLLMs rely on System 1 reason-